

LPOO — V2: Memória, Pegadinhas e Recuperação

Objetivo desta versão: estudar para a prova sem depender de lembrar tudo de uma vez. Cada regra é transformada em uma sequência de memória: **ERRO** → **CONSEQUÊNCIA** → **REGRA** → **RESULTADO**.

Como estudar este MD

1. Leia somente a **Regra**.
2. Cubra o Resultado e tente lembrar.
3. Leia o **Erro** como uma pegadinha que você não quer repetir.
4. Faça o exemplo sem olhar.
5. Só então confira o resultado.

Frase de segurança: “Eu não preciso lembrar tudo. Preciso reconhecer a pegadinha e aplicar a regra.”

Modo “DEU MERDA”

A ideia é criar uma memória forte a partir de um **erro fictício de exercício**, não de culpa real. O cérebro encontra o erro → percebe a consequência → recupera a regra → chega ao resultado.

ERRO	CONSEQUÊNCIA	REGRA	RESULTADO
Tratar private como inútil	Tornar tudo public	Campo fica protegido; acesso passa pela API	private + getter/setter
Esquecer que Java não herda duas classes	Tentar extends A, B	Uma classe estende uma classe	Várias capacidades → interfaces
Colocar super depois de outra instrução	Erro de compilação no construtor	super(...) deve iniciar o construtor	Filha chama mãe primeiro
Confundir com overload	Achar que parâmetros podem mudar	Override mantém a assinatura	@Override ajuda o compilador
Mudar apenas retorno	Esperar duas assinaturas pelo retorno	Parâmetros precisam diferir	f() e f(int)
Olhar apenas o tipo da variável	Achar que Livro 1 chama sempre Livro	Objeto concreto decide override em execução	Livro l=new LivroDigital(...);
Instanciar a classe que deveria ser abstrata	Permitir comportamento falso	abstract impede instância direta	Forma não deve representar área zero genérica
Tratar interface como classe mãe comum	Usar construtor/estado de instância nela	Interface define contrato	Carro implements Alugavel
Usar herança porque existe relação qualquer	Criar hierarquia desnecessária	‘tem um’ → composição	Livro tem CodigoLivro
Achar que cada objeto possui seu próprio contador	Valores não compartilhados	static pertence à classe	contador é compartilhado
Achar que final congela conteúdo de objeto	Tentar entender List final como imutável	final congela a referência	final List ainda pode add
Lançar e nunca entender o fluxo	Programa encerra com stack trace	try/catch trata exceções; finally ajuda na limpeza	catch específico antes do genérico

1. Regras que você precisa reconhecer rápido

1. Encapsulamento

Erro: Tratar private como inútil

Consequência: Tornar tudo public

Regra: Campo fica protegido; acesso passa pela API

Resultado: private + getter/setter

2. Herança

Erro: Esquecer que Java não herda duas classes

Consequência: Tentar extends A, B

Regra: Uma classe estende uma classe

Resultado: Várias capacidades → interfaces

3. super

Erro: Colocar super depois de outra instrução

Consequência: Erro de compilação no construtor

Regra: super(...) deve iniciar o construtor

Resultado: Filha chama mãe primeiro

4. Override

Erro: Confundir com overload

Consequência: Achar que parâmetros podem mudar

Regra: Override mantém a assinatura

Resultado: @Override ajuda o compilador

5. Overload

Erro: Mudar apenas retorno

Consequência: Esperar duas assinaturas pelo retorno

Regra: Parâmetros precisam diferir

Resultado: f() e f(int)

6. Polimorfismo

Erro: Olhar apenas o tipo da variável

Consequência: Achar que Livro l chama sempre Livro

Regra: Objeto concreto decide override em execução

Resultado: Livro l=new LivroDigital(...);

7. Abstração

Erro: Instanciar a classe que deveria ser abstrata

Consequência: Permitir comportamento falso

Regra: abstract impede instância direta

Resultado: Forma não deve representar área zero genérica

8. Interface

Erro: Tratar interface como classe mãe comum

Consequência: Usar construtor/estado de instância nela

Regra: Interface define contrato

Resultado: Carro implements Alugavel

9. Composição

Erro: Usar herança porque existe relação qualquer

Consequência: Criar hierarquia desnecessária

Regra: ‘tem um’ $\rightarrow$ composição

Resultado: Livro tem CodigoLivro

10. static

Erro: Achar que cada objeto possui seu próprio contador

Consequência: Valores não compartilhados

Regra: static pertence à classe

Resultado: contador é compartilhado

11. final

Erro: Achar que final congela conteúdo de objeto

Consequência: Tentar entender List final como imutável

Regra: final congela a referência

Resultado: final List ainda pode add

12. Exceção

Erro: Lançar e nunca entender o fluxo

Consequência: Programa encerra com stack trace

Regra: try/catch trata exceções; finally ajuda na limpeza

Resultado: catch específico antes do genérico

13. equals/hashCode

Erro: Comparar objetos pelos dados sem sobrescrever equals

Consequência: remove/contains falham

Regra: equals e hashCode precisam ser coerentes

Resultado: Objetos iguais → mesmo hash

14. enum

Erro: Guardar status como texto livre

Consequência: Digitação errada passa

Regra: enum restringe valores

Resultado: PENDENTE/PAGA/CANCELADA

15. instanceof

Erro: Usar em todo lugar

Consequência: Código vira cascata de tipos

Regra: Use quando a verificação de tipo é realmente necessária

Resultado: instanceof LivroDigital

2. Revisão relâmpago antes da prova

- **Encapsulamento:** Campo fica protegido; acesso passa pela API → private + getter/setter
- **Herança:** Uma classe estende uma classe → Várias capacidades → interfaces
- **super:** super(...) deve iniciar o construtor → Filha chama mãe primeiro
- **Override:** Override mantém a assinatura → @Override ajuda o compilador
- **Overload:** Parâmetros precisam diferir → f() e f(int)
- **Polimorfismo:** Objeto concreto decide override em execução → Livro l=new LivroDigital(...);
- **Abstração:** abstract impede instância direta → Forma não deve representar área zero genérica
- **Interface:** Interface define contrato → Carro implements Alugavel
- **Composição:** ‘tem um’ → composição → Livro tem CodigoLivro
- **static:** static pertence à classe → contador é compartilhado

3. Método de recuperação ativa

Quando bater o branco, não releia a matéria inteira. Faça a sequência:

1. O que estou tentando descobrir?
2. Qual é a pegadinha mais provável?
3. Qual regra mata essa pegadinha?
4. Qual resultado essa regra produz?
5. Consigo explicar em uma frase?

4. Sessões curtas

Use blocos de 15–25 minutos. Em cada bloco, escolha poucos itens, tente lembrar sem consultar e marque os erros. No próximo bloco, revise principalmente os que você errou.

5. Regra contra o esquecimento

Não tente decorar a página inteira. Transforme cada conteúdo em uma associação curta:

PEGADINHA → REGRA → EXEMPLO → RESULTADO.

Isso permite reconstruir a resposta mesmo quando a memória da frase original desaparece.